

POP para Extração e Análise de Dados em Sistemas de Informação via Banco de Dados MySQL: no Contexto de Atuação do Engenheiro de Produção

Gustavo Gonçalves Barcelos (Universidade Federal de Goiás - UFG)

João Vitor Bueno dos Reis (Universidade Federal de Goiás - UFG)



Este artigo apresenta um Procedimento Operacional Padrão (POP) para acesso, extração e análise de dados diretamente em banco de dados relacional MySQL no contexto da Engenharia de Produção. A motivação central é a necessidade de autonomia analítica do engenheiro de produção, que historicamente recebe dados filtrados, atrasados e agregados por camadas intermediárias de TI. O trabalho fundamenta-se nos conceitos de Sistemas de Informação Gerencial (SIG), arquitetura de software em camadas e modelagem de bancos de dados relacionais, articulando o ciclo dado → informação → conhecimento com a prática de consultas SQL. São abordados normalização de dados (1FN, 2FN e 3FN), tipos de relacionamento (1:1, 1:N, N:M), além das diferenças entre bancos relacionais e NoSQL. O procedimento documentado descreve as etapas de conexão ao MySQL, escrita de queries e geração de indicadores operacionais, usando como contexto ilustrativo uma fábrica de componentes automotivos. Os resultados esperados compreendem a capacitação do engenheiro para responder perguntas analíticas com dados primários e atualizados, reduzindo a dependência de relatórios prontos.

Palavras-chave: Banco de dados relacional; MySQL; SQL; Engenharia de Produção; SIG; procedimento operacional padrão; análise de dados.

1. Introdução

A transformação digital das organizações produtivas tem colocado o dado como um dos principais ativos estratégicos da empresa moderna. Schwab (2016), ao caracterizar a Quarta Revolução Industrial, destaca que a convergência entre sistemas físicos e digitais, por meio de tecnologias como sistemas ERP, MES e WMS, computação em nuvem e inteligência artificial, gera volumes sem precedentes de dados operacionais, exigindo das organizações novas capacidades analíticas para transformar esses dados em decisões. Nesse cenário, a gestão eficaz da informação deixa de ser uma função exclusiva da área de Tecnologia da Informação e passa a ser uma competência essencial de todas as áreas funcionais da empresa, incluindo a Engenharia de Produção.

Laudon e Laudon (2022) definem um Sistema de Informação como um conjunto de componentes inter-relacionados que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisão e o controle em uma organização. Os autores ressaltam, contudo, que a maioria dos profissionais acessa essa informação por camadas intermediárias, relatórios pré-formatados, planilhas exportadas e dashboards com escopo predefinido, o que implica perda de granularidade, atraso temporal e viés de seleção sobre os dados apresentados. Davenport e Harris (2007), em seu estudo sobre organizações orientadas por análise, demonstram que as empresas que constroem vantagem competitiva a partir de dados são precisamente aquelas cujos profissionais operacionais têm acesso direto às fontes primárias de informação, sem depender exclusivamente de relatórios gerados por terceiros.

No âmbito da Engenharia de Produção, essa limitação é particularmente crítica. Slack, Brandon-Jones e Johnston (2018) argumentam que a gestão de operações exige monitoramento contínuo de indicadores de desempenho, como eficiência de linha, tempo de ciclo, nível de estoque e taxa de defeitos, e que a qualidade dessas decisões depende diretamente da atualidade e da precisão dos dados disponíveis. Quando o Engenheiro de Produção depende de relatórios elaborados por outra área, o dado chega filtrado, atrasado e agregado demais para sustentar análises de causa raiz ou simulações de cenário. Corrêa e Corrêa (2017) reforçam essa perspectiva ao apontar que a integração entre sistemas de informação e decisões operacionais é um dos pilares da gestão moderna da produção, e que a capacidade de interpretar dados diretamente dos sistemas transacionais amplia significativamente a autonomia do gestor de produção.

A formalização de procedimentos de acesso a dados como Procedimentos Operacionais Padrão (POP) é respaldada pela literatura de qualidade e gestão de processos. A ABNT NBR ISO 9001:2015 estabelece que a organização deve manter informação documentada na extensão necessária para apoiar a operação de seus processos e reter informação documentada para ter confiança de que os processos foram realizados conforme planejado (ABNT, 2015). Nesse sentido, um POP que descreva sistematicamente o procedimento de extração e análise de dados em bancos de dados atende a dois requisitos simultâneos: garante a repetibilidade e a rastreabilidade do processo analítico, e reduz a variabilidade nos resultados gerados por diferentes analistas.

Do ponto de vista técnico, Date (2004) define um banco de dados como um repositório de dados persistentes, utilizado pelos sistemas de aplicação de uma determinada empresa, e destaca que o modelo relacional, proposto originalmente por Codd (1970), oferece independência de dados, integridade referencial e uma linguagem declarativa (SQL) que

permite ao usuário final expressar o que deseja obter sem precisar especificar como o sistema deve obtê-lo. Essa característica torna o SQL uma ferramenta acessível a profissionais não especializados em programação, desde que devidamente capacitados, o que inclui o Engenheiro de Produção que necessita realizar análises operacionais de forma autônoma.

Diante desse contexto, este trabalho justifica-se pela necessidade de formalizar o processo de acesso direto a bancos de dados como uma prática profissional estruturada e replicável no âmbito da Engenharia de Produção. A questão central que orienta a pesquisa é: de que forma um Engenheiro de Produção pode estruturar um Procedimento Operacional Padrão para extração e análise de dados operacionais diretamente em um banco de dados relacional?

O objetivo geral é elaborar um POP, no formato de artigo científico segundo as normas do ENEGEP (ABEPRO), que descreva o procedimento de acesso, extração e análise de dados em um banco de dados real, aplicado ao contexto da Engenharia de Produção. Os objetivos específicos são: (i) revisar os fundamentos teóricos de Sistemas de Informação Gerencial, arquitetura de software e bancos de dados; (ii) apresentar a modelagem e normalização de dados do cenário escolhido; (iii) documentar o procedimento de conexão e as queries utilizadas; e (iv) gerar indicadores de desempenho operacional a partir dos dados extraídos.

2. Fundamentação Teórica

2.1 Sistemas de Informação Gerencial (SIG) e lógica analítica

Os Sistemas de Informação Gerencial (SIG) são responsáveis por transformar dados operacionais em informações úteis para apoio à tomada de decisão. Esses sistemas integram dados provenientes de diferentes áreas da organização, como produção, estoque e qualidade, permitindo análise estruturada dos processos (LAUDON; LAUDON, 2020).

No contexto da Engenharia de Produção, os SIG estão diretamente associados a sistemas como ERP, MES e WMS, que registram continuamente eventos do chão de fábrica. Entretanto, o valor desses sistemas não está apenas no armazenamento dos dados, mas na capacidade de transformá-los em conhecimento aplicável.

Esse processo pode ser compreendido pelo modelo apresentado no Quadro 1.

Quadro 1 — Transformação de dados em conhecimento

Nível	Descrição	Exemplo em EP
Dado	Registro bruto, sem contexto	“150 unidades em estoque”
Informação	Dado interpretado em contexto	“Estoque abaixo do mínimo”
Conhecimento	Informação aplicada à decisão	“Necessidade de reposição imediata”

Fonte: Adaptado de Laudon e Laudon (2020).

Assim, observa-se que o banco de dados atua como base para geração de informação, enquanto as consultas e análises representam o mecanismo de transformação em conhecimento. Nesse sentido, a capacidade de acessar diretamente os dados amplia significativamente o potencial analítico do engenheiro.

2.2. Arquitetura de Software e o Papel do Banco de Dados

2.2.1 Arquitetura em Camadas (N-Tier)

A arquitetura em camadas (N-Tier) é um dos padrões mais consolidados no desenvolvimento de sistemas corporativos. Seu princípio fundamental é a separação de responsabilidades: cada camada possui uma função específica e comunica-se apenas com as camadas adjacentes, reduzindo o acoplamento e facilitando a manutenção e a evolução do sistema (PRESSMAN; MAXIM, 2020). No modelo de três camadas clássico, encontrado em ERPs e sistemas MES industriais, a organização é a seguinte: (i) camada de apresentação, responsável pela interface com o usuário, dashboards, relatórios, telas de apontamento; (ii) camada de aplicação ou negócio, onde residem as regras da organização, cálculo de OEE, sequenciamento de ordens, validações de qualidade; e (iii) camada de dados, responsável pelo armazenamento persistente e recuperação das informações, o banco de dados.

Essa separação tem implicações diretas para o engenheiro de produção: os relatórios e dashboards que ele consulta diariamente são produtos da camada de apresentação, que recebe dados já filtrados e agregados pelas camadas superiores. Acessar o banco diretamente significa bypassar essas camadas intermediárias e trabalhar com os dados em seu estado mais bruto e completo.

2.2.3 Arquitetura Hexagonal (Ports and Adapters)

Proposta por Alistair Cockburn em 2005, a arquitetura hexagonal (também chamada Ports and Adapters) representa uma evolução em relação ao MVC tradicional. Seu princípio central é proteger o núcleo de negócio da aplicação de dependências externas, como banco de dados, interfaces de usuário e serviços de mensageria, por meio de portas (interfaces abstratas) e adaptadores (implementações concretas) (COCKBURN, 2005).

Nesse modelo, o banco de dados é explicitamente tratado como um detalhe de infraestrutura: o núcleo da aplicação define uma interface de repositório (por exemplo, `OrdemProducaoRepository`), e o adaptador concreto implementa essa interface para o MySQL, PostgreSQL ou MongoDB. Isso significa que, do ponto de vista arquitetural, trocar o SGBD não exige reescrever a lógica de negócio, apenas substituir o adaptador. Para o engenheiro de produção, isso reforça que o banco de dados é uma peça substituível e que o conhecimento de SQL transcende qualquer SGBD específico.

2.2.4 Clean Architecture e o Princípio da Inversão de Dependência

A Clean Architecture, sistematizada por Robert C. Martin (2017), organiza o software em círculos concêntricos de responsabilidade, com as regras de negócio no centro e os detalhes de infraestrutura na periferia. A regra fundamental é que as dependências sempre apontam para dentro: camadas externas (banco de dados, UI, frameworks) dependem das internas (casos de uso, entidades), nunca o contrário.

Esse princípio tem consequência direta na forma como o banco de dados é visto: ele não é o coração do sistema, mas um plugin substituível. Os dados que o engenheiro de produção precisa existem independentemente do SGBD que os armazena, o que importa é a capacidade

de formular as perguntas corretas e extrair as respostas. O Quadro 2 resume as principais arquiteturas e como cada uma posiciona o banco de dados no sistema.

Quadro 2 — Principais Padrões Arquiteturais e o Papel do Banco de Dados

Padrão	Origem	Papel do BD	Aplicação em EP
N-Tier (camadas)	Anos 1990	Camada de dados isolada	ERP, MES, WMS corporativos
MVC	Reenskaug (1979)	Model — acessa e persiste dados	Sistemas web de gestão de produção
Hexagonal	Cockburn (2005)	Adaptador de infraestrutura substituível	Sistemas com múltiplos SGBDs
Clean Architecture	Martin (2017)	Plugin periférico, detalhe de implementação	Sistemas de alto desacoplamento

Fonte: elaborado pelo autor com base em Fowler (2002) e Martin (2017).

2.3 Bancos de dados e organização da informação

2.3.1 Modelo relacional

O modelo relacional, proposto por Codd (1970), organiza os dados em tabelas compostas por linhas (registros) e colunas (atributos). Cada tabela possui uma chave primária responsável por identificar unicamente cada registro, enquanto os relacionamentos entre tabelas são realizados por meio de chaves estrangeiras (ELMASRI; NAVATHE, 2011).

Esse modelo é amplamente utilizado em sistemas industriais devido à sua consistência e facilidade de consulta por meio da linguagem SQL.

2.3.2 Normalização de dados

A normalização é um processo que visa organizar os dados de forma a eliminar redundâncias e evitar inconsistências. Esse processo é guiado pelas chamadas formas normais (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Quadro 3 — Formas normais básicas

Forma Normal	Objetivo principal	Problema evitado
1FN	Eliminar atributos não atômicos	Campos com múltiplos valores
2FN	Remover dependências parciais	Dados duplicados
3FN	Eliminar dependências transitivas	Inconsistência de dados

Fonte: Adaptado de Silberschatz, Korth e Sudarshan (2020).

A aplicação dessas regras resulta em bancos de dados mais organizados, facilitando tanto a manutenção quanto a análise dos dados.

2.3.3 Relacionamentos entre tabelas

Os relacionamentos estruturam a forma como as entidades se conectam dentro do banco de dados. Os principais tipos são apresentados no Quadro 4.

Quadro 4 — Tipos de relacionamento

Tipo	Descrição	Exemplo em EP
1:1	Um registro relaciona-se com outro único	Máquina ↔ ficha técnica
1:N	Um registro relaciona-se com vários	Fornecedor → pedidos
N:M	Muitos para muitos (tabela intermediária)	Produtos ↔ ordens de produção

Fonte: Adaptado de Elmasri e Navathe (2011).

Esses relacionamentos são essenciais para representar a estrutura real dos sistemas produtivos, permitindo análises integradas entre diferentes áreas.

2.3.4 SQL como ferramenta analítica

A linguagem SQL é o principal meio de interação com bancos de dados relacionais, permitindo realizar consultas, filtragens e agregações de dados (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). Por meio de operações como JOIN e funções de agregação (SUM, AVG, COUNT), é possível gerar indicadores diretamente no banco de dados, reduzindo a dependência de ferramentas externas. No contexto da Engenharia de Produção, isso possibilita análises de: produtividade por linha de produção; níveis de estoque; desempenho de fornecedores e indicadores de qualidade.

2.3.5. MySQL

O MySQL é um SGBD relacional open source desenvolvido originalmente pela empresa sueca MySQL AB e atualmente mantido pela Oracle Corporation. É o banco de dados relacional mais amplamente utilizado no mundo, especialmente em aplicações web, sistemas SaaS e ERPs de médio porte (MYSQL, 2026). Sua arquitetura modular permite a escolha de diferentes engines de armazenamento: o InnoDB, engine padrão desde a versão 5.5, oferece suporte completo a transações ACID, chaves estrangeiras e recuperação automática após falhas, características essenciais para ambientes de produção industrial.

No contexto da Engenharia de Produção, o MySQL é frequentemente encontrado como backend de sistemas MES de pequeno e médio porte, sistemas de gestão de qualidade e plataformas de controle de estoque. Sua facilidade de instalação, ampla documentação e suporte a Python (via mysql-connector-python) e a ferramentas gráficas como MySQL Workbench e DBeaver fazem dele uma escolha natural para o primeiro contato do engenheiro com bancos de dados relacionais.

2.3.6. PostgreSQL

O PostgreSQL é um SGBD relacional open source considerado o mais avançado de sua categoria em conformidade com o padrão SQL e em suporte a tipos de dados complexos (POSTGRESQL, 2026). Originado no projeto POSTGRES da Universidade da Califórnia em Berkeley nos anos 1980, o PostgreSQL destaca-se por recursos que o MySQL não oferece nativamente: tipos de dados avançados como JSON nativo com indexação, arrays, tipos geométricos e GIS (Geographic Information System); suporte a expressões de tabela comuns (CTEs) recursivas; funções de janela (window functions) para análises temporais; e conformidade estrita com o padrão ANSI SQL.

Para a Engenharia de Produção, o PostgreSQL é a escolha recomendada para sistemas analíticos robustos e para cenários que envolvem dados georreferenciados, como rastreamento de frotas de entrega ou mapeamento de plantas industriais. Sua capacidade de armazenar e indexar documentos JSON sem abrir mão das garantias relacionais o posiciona como uma solução híbrida entre o mundo relacional e o NoSQL. O Quadro 5 sintetiza a comparação entre MySQL e PostgreSQL.

Quadro 5 — MySQL vs. PostgreSQL: Comparativo Técnico

Característica	MySQL	PostgreSQL
Licença	GPL / Oracle	PostgreSQL License (open source)
Conformidade SQL	Moderada	Alta (ANSI SQL)
JSON nativo	Suporte básico	JSONB com indexação avançada
Window Functions	Parcial	Completo
Tipos avançados	Limitado	Arrays, GIS, tipos customizados
Performance leitura	Excelente (simples)	Excelente (consultas complexas)
Uso típico em EP	MES, estoque, web	Analytics, BI, dados geoespaciais

Fonte: elaborado pelos autores com base em MySQL (2026) e PostgreSQL (2026).

2.3.7. MongoDB

O MongoDB é o principal representante dos bancos de dados orientados a documentos. Lançado em 2009 pela empresa 10gen (atual MongoDB Inc.), armazena dados em documentos BSON (Binary JSON), uma representação binária do formato JSON, sem impor um esquema rígido às coleções (MONGODB, 2026). Essa flexibilidade de schema (schema-less) permite que cada documento em uma coleção tenha uma estrutura diferente, o que é particularmente útil em cenários onde os dados variam por tipo de equipamento ou por linha de produção.

No contexto industrial, o MongoDB é frequentemente empregado para o armazenamento de leituras de sensores IoT, onde diferentes equipamentos expõem diferentes conjuntos de métricas. Um torno CNC pode reportar temperatura do spindle, vibração e corrente elétrica, enquanto uma injetora de plástico reporta pressão de injeção, temperatura do molde e tempo

de ciclo, estruturas incompatíveis em um banco relacional sem recorrer a colunas nulas ou tabelas genéricas.

O pipeline de agregação do MongoDB é o equivalente funcional do GROUP BY + JOIN do SQL, permitindo filtrar, desagrupar arrays embutidos, agrupar por campos e calcular métricas em uma única operação encadeada. Para análises de séries temporais de sensores, o MongoDB 5.0 introduziu as Time Series Collections, otimizadas para ingestão e compressão de dados temporais com alta frequência.

2.3.8 Redis

O Redis (Remote Dictionary Server) é um banco de dados do tipo chave-valor (key-value store) que opera primariamente em memória RAM, com persistência opcional em disco (REDIS, 2026). Desenvolvido por Salvatore Sanfilippo em 2009, o Redis suporta estruturas de dados ricas além do simples par chave-valor: strings, listas, conjuntos (sets), conjuntos ordenados (sorted sets), hashes e streams, o que o diferencia de outros key-value stores mais simples.

Na Engenharia de Produção, o Redis é aplicado em três cenários principais: (i) cache de consultas, para armazenar em memória os resultados de queries frequentes ao banco relacional, reduzindo o tempo de resposta de dashboards de produção em tempo real; (ii) filas de mensagens, gerenciar a fila de eventos entre sensores IoT e o sistema de processamento, garantindo que nenhuma leitura seja perdida em picos de produção; e (iii) sessões e estado temporário, manter o estado de processos de automação que precisam de acesso ultrarrápido a variáveis intermediárias. O Redis não é adequado como banco de dados primário para dados transacionais, mas como componente de infraestrutura complementar é indispensável em arquiteturas de alto desempenho.

2.3.9 Apache Cassandra

O Apache Cassandra é um banco de dados colunar distribuído, projetado para oferecer alta disponibilidade e escalabilidade linear sem ponto único de falha (CASSANDRA, 2026). Desenvolvido originalmente pelo Facebook em 2008 e doado à Apache Software Foundation, o Cassandra adota uma arquitetura peer-to-peer onde todos os nós são equivalentes, não há nó mestre e os dados são replicados automaticamente entre múltiplos nós e data centers.

A linguagem de consulta do Cassandra, o CQL (Cassandra Query Language), tem sintaxe similar ao SQL, mas com restrições importantes: não suporta JOINS nem subqueries, e o modelo de dados deve ser projetado orientado às queries o oposto do modelo relacional, onde se modela os dados e depois se define as queries. Essa característica exige um planejamento mais cuidadoso do schema, mas resulta em performance de leitura e escrita previsível e altíssima mesmo com bilhões de registros.

Para a Engenharia de Produção, o Cassandra é o banco ideal para séries temporais de alta frequência: leituras de sensores a cada segundo em centenas de máquinas distribuídas por múltiplas plantas, logs de eventos de máquinas CNC, registros de telemetria de frotas e históricos de alarmes de sistemas SCADA. O Quadro 6 apresenta uma visão consolidada dos quatro SGBDs discutidos.

Quadro 6 — Comparativo Geral dos SGBDs para Engenharia de Produção

SGBD	Tipo	Ponto forte	Limitação principal	Caso de uso em EP
MySQL	Relacional	Simplicidade, ecossistema amplo	Conformidade SQL limitada	MES, estoque, qualidade
PostgreSQL	Relacional	Conformidade SQL, tipos avançados	Configuração mais complexa	Analytics, BI, dados geoespaciais
MongoDB	Documental	Schema flexível, pipeline de agregação	Sem JOINS nativos eficientes	Sensores IoT, catálogos variáveis
Redis	Chave-Valor	Latência sub-milissegundo	Dados limitados pela RAM	Cache, filas, estado em tempo real
Cassandra	Colunar distribuído	Escalabilidade linear, sem falha única	Sem JOINS, modelo query-first	Telemetria, séries temporais massivas

Fonte: elaborado pelos autores (2026).

2.3.10. Polyglot Persistence: Usando o BD Certo para Cada Problema

A estratégia polyglot persistence, termo cunhado por Sadalage e Fowler (2012), preconiza que sistemas complexos devem utilizar múltiplos tipos de banco de dados simultaneamente, cada um aplicado ao problema para o qual foi otimizado, em vez de forçar todos os dados em um único SGBD. Essa abordagem rompe com o paradigma do banco de dados único e universal, reconhecendo que cada tecnologia tem trade-offs e que a escolha deve ser guiada pela natureza dos dados e pelo padrão de acesso.

Em uma planta industrial moderna com arquitetura polyglot, uma configuração típica seria: MySQL ou PostgreSQL para os dados transacionais estruturados do ERP, ordens de produção, notas fiscais, movimentações de estoque, onde as garantias ACID e a integridade referencial são inegociáveis; MongoDB para os registros de sensores IoT com estrutura variável por tipo de equipamento; Redis como cache de dashboard em tempo real e fila de eventos entre o chão de fábrica e o sistema de processamento; e Cassandra para o histórico de séries temporais de longa duração, onde bilhões de leituras precisam ser armazenadas e consultadas com alta performance. O Quadro 7 ilustra essa configuração.

Quadro 7 — Exemplo de Arquitetura Polyglot em Planta Industrial

Dado	Volume	SGBD escolhido	Justificativa
Ordens de produção e estoque	Médio (milhões)	PostgreSQL	ACID, integridade referencial, analytics

Leituras de sensores IoT	Alto (schema variável)	MongoDB	Flexibilidade de schema por equipamento
Cache de KPIs do dashboard	Pequeno (memória)	Redis	Latência < 1ms, atualização em tempo real
Histórico de telemetria	Massivo (bilhões)	Cassandra	Escalabilidade horizontal, séries temporais

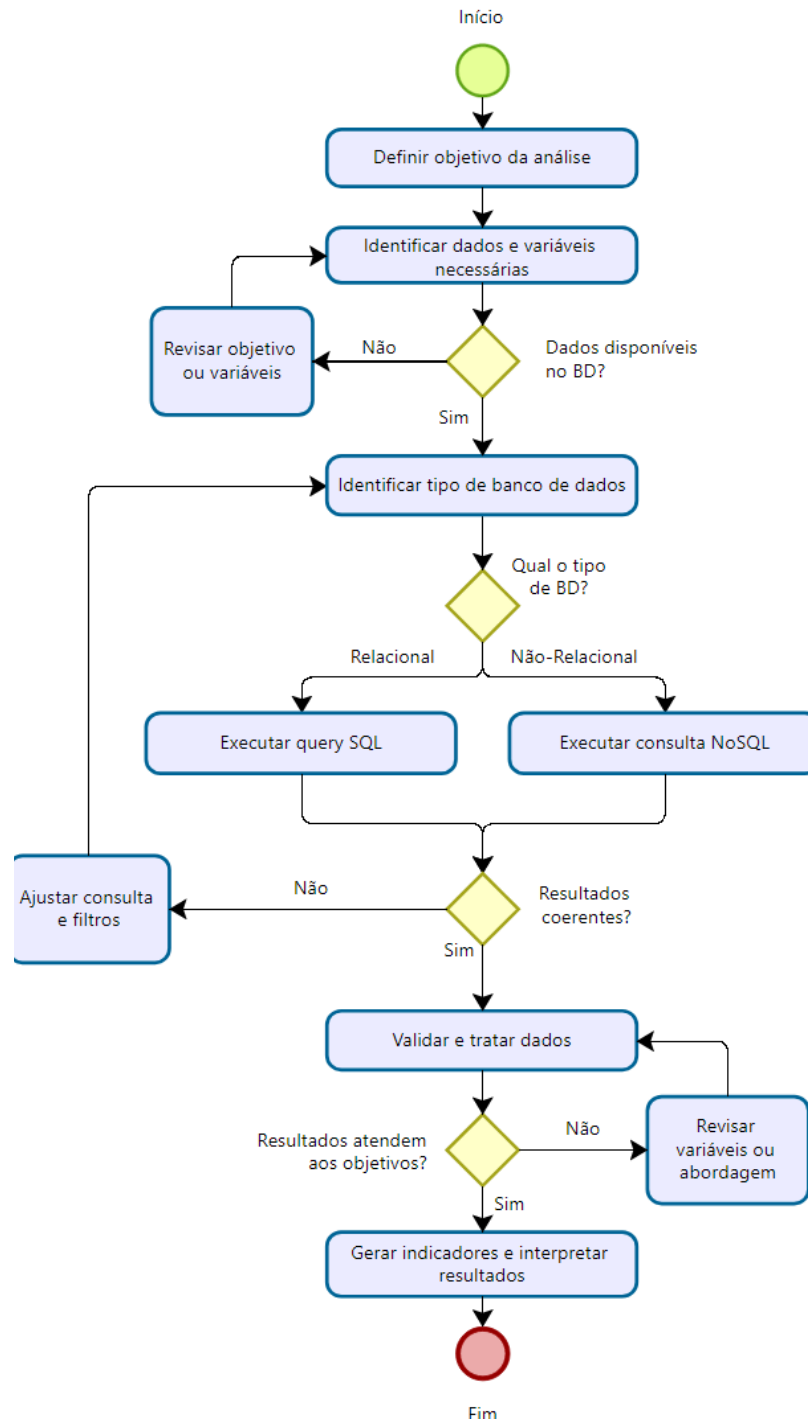
Fonte: elaborado pelo autor com base em Sadalage e Fowler (2012).

3. Metodologia

3.1 Processo de conexão com o banco de dados

A metodologia baseia-se na construção de um Procedimento Operacional Padrão (POP) para acesso e análise de dados em qualquer banco de dados, se limitando, nesse caso, ao MySQL; PostgreSQL; MongoDB e Cassandra, estruturado como um processo iterativo. O fluxo incorpora pontos de decisão que permitem a identificação do banco de dados a ser acessado e o refinamento contínuo da análise, garantindo consistência e aderência aos objetivos propostos.

Figura 1 - Fluxograma do POP de acesso e análise à banco de dados



Fonte: Elaborado pelos autores (2026)

A aplicação do Procedimento Operacional Padrão (POP) para acesso e análise de dados em sistemas de informação segue o fluxo apresentado na Figura 1, no qual cada etapa representa uma fase do processo analítico, incluindo pontos de decisão e ciclos de refinamento.

Inicialmente, realiza-se a definição do objetivo da análise, etapa responsável por direcionar todo o processo. Em seguida, ocorre a identificação dos dados e variáveis necessárias, considerando quais informações são requeridas para atender ao objetivo proposto. Na sequência, verifica-se a disponibilidade dos dados no banco de dados. Caso os dados não

estejam disponíveis, o fluxo retorna à etapa anterior, permitindo a revisão do objetivo ou das variáveis consideradas.

Uma vez confirmada a disponibilidade dos dados, realiza-se a identificação do tipo de banco de dados utilizado pelo sistema de informação. Essa identificação pode ocorrer por meio das configurações de conexão, portas de serviço ou respostas fornecidas pelo próprio servidor, permitindo distinguir bancos relacionais, como MySQL e PostgreSQL, de bancos não relacionais, como MongoDB e Cassandra.

Após essa identificação, procede-se à conexão com o banco de dados e ao mapeamento de sua estrutura. Em bancos relacionais, essa etapa envolve a exploração de tabelas, atributos e relacionamentos por meio de comandos como *SHOW TABLES* e *DESCRIBE*. Já em bancos NoSQL, o processo envolve a identificação de coleções, documentos ou estruturas distribuídas. O entendimento da organização dos dados é essencial para garantir a correta construção das consultas.

Com base nesse mapeamento, inicia-se a etapa de construção e execução das consultas. Em bancos relacionais, utilizam-se comandos SQL como *SELECT*, *WHERE*, *JOIN* e *GROUP BY*, enquanto bancos NoSQL utilizam mecanismos específicos de consulta e agregação. Caso os resultados obtidos não sejam coerentes, o fluxo retorna à própria etapa de construção das consultas, permitindo ajustes em filtros, junções, agregações ou estruturas de busca.

Após a obtenção de resultados consistentes, realiza-se a validação e tratamento dos dados, verificando aspectos como duplicidade, integridade e coerência com o contexto operacional. Na sequência, avalia-se se os resultados atendem ao objetivo da análise. Caso não atendam, o fluxo retorna à etapa de identificação das variáveis ou abordagem analítica, possibilitando o refinamento do processo.

Quando os resultados são satisfatórios, procede-se à geração de indicadores e interpretação dos resultados, etapa em que os dados são transformados em informações úteis para apoio à tomada de decisão. Por fim, todo o processo é documentado na forma de um Procedimento Operacional Padrão (POP), incluindo consultas utilizadas, parâmetros de acesso e finalidade da análise. Essa documentação garante reprodutibilidade, rastreabilidade e padronização das análises futuras.

8. CONSIDERAÇÕES FINAIS

Este trabalho apresentou os fundamentos teóricos e o procedimento operacional necessários para que o engenheiro de produção acesse, extraia e analise dados diretamente em um banco de dados MySQL. A fundamentação em Sistemas de Informação Gerencial demonstrou que o banco de dados é a camada primária de toda informação organizacional, e que a capacidade de acessá-lo diretamente representa um diferencial competitivo concreto frente à dependência de relatórios intermediários.

A discussão sobre normalização (1FN, 2FN e 3FN) e tipos de relacionamento evidenciou que um modelo de dados bem estruturado é condição necessária para a confiabilidade das consultas. A comparação entre abordagens relacionais e NoSQL, culminando na estratégia polyglot persistence, posicionou o MySQL como a escolha adequada para dados transacionais estruturados, como ordens de produção, estoques e registros de qualidade, enquanto bancos NoSQL complementam o ecossistema para dados de sensores e telemetria.

Como próximos passos, a parte prática (N2) deste trabalho prevê a conexão a um banco de dados real ou ao dataset disponibilizado pelo professor, com execução das queries documentadas neste POP, geração de indicadores de Engenharia de Produção e análise crítica dos resultados. O objetivo final é demonstrar, com evidências concretas, que o engenheiro de produção que domina dados toma decisões melhores e mais rápidas.

REFERÊNCIAS

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Campus, 2004.

ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson, 2011.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Sistema de Banco de Dados. 7. ed. Rio de Janeiro: GEN LTC, 2020.

STAIR, R. M.; REYNOLDS, G. W. Princípios de Sistemas de Informação. 11. ed. São Paulo: Cengage, 2015.

MySQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: mai. 2026.

FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

MONGODB. MongoDB Manual. Disponível em: <https://www.mongodb.com/docs/manual/>. Acesso em: mai. 2026.